Speaker Script and Live Demo Run Book · v3

BlocBuy — Team 8 · Wednesday 21 October 2026, 16:15–16:55 SGT (Zoom)

Responsible	Lim Jiechao (presentation lead, #461); reviewer Liu Jin. v3 builds on the v2 deck and script, with the team's v3 change document applied and every `[VERIFY]` item checked in the repository.
Companion files	`swe5001-presentation-v3.html`: the deck, 24 slides plus 17 backup slides. Open it in Chrome or Edge. Press **S** for the notes window with this script and a pace timer, **B** for the backup index, **O** for the overview, **F** for fullscreen. `swe5001-presentation-v3.pdf` is the same deck as a PDF, typeset with Typst from `swe5001-presentation-v3.typ`. The v2 files are unchanged.
Single source	`build/deck-content-v3.js` holds the wording, timings, cues and the 19 Oct results; `build/slides-v3.js` holds the layouts; `build/diagrams/` holds the UML diagrams. To rebuild, run `node build-html-v3.js && bash export-typst-v3.sh` from `build/`, then recompile this file.
Facts as of	19 Oct 2026. Everything that depends on the 19 Oct session is in one block at the top of the content file. It is now final (`final: true`): V3 ran on AWS; every other test is shown as "not run on AWS" with its local result. No gaps remain.

How to use this document

- **Speakers:** read your segment (§1–§5) aloud against a timer at least twice. Each slide card shows the slot, what changed in v3, and how long the sample wording takes at 130 words a minute. Every card now fits its slot. Rephrase in your own words; never read the slide.
- **Demo team:** §6 is the run book: preconditions, screen layout, then every step with what to operate, what to say, what you should see and when to fall back. §7 adds Plan C.
- **Lead:** the run sheet and coverage checks below, the contingencies in §7, the timeline in §8 and the checks and open items in §9 are yours.

Structure follows "04 Project Presentation Guidelines & Schedule" V2.8 exactly: one chapter per segment (1 Overview, 2 Project Conduct, 3 Logical, 4 Physical, 5 DevOps, 6 Demo), each opening with the guideline's own sub-items.

What changed from v2

Slides	Change	Why
All	New colour design (charcoal, red, blue and amber on cream; Inter); simpler wording; issue numbers and course codes kept off the slides	Readability on Zoom
3, 4	Future seeds line; payments as the documented next step	Success criterion “more seeds in future”
5	Three columns: verified 17 Oct, tested 19 Oct, still open	Guideline 2a: honest status
6	Actual milestone dates (17 Oct release added, “code freeze” removed); Lim’s and Eric’s roles expanded	Accuracy
7, 8, 12	Small decision tags (AD-02 … AD-14) so the report can cross-reference	Traceability
9, 13, 16	UML diagrams from the team replace the v2 box diagrams; the v2 diagrams are backups B13–B15	Guideline 3b, 4b, 4d
11	Measured reuse (audit queue: 2 lines of Terraform, 0 commerce changes) and the public API contract	Briefing Q2, Q3; “robust API access”
12	Measured cost: US$0.49 for 7 sessions	Briefing Q6
14	Capacity statement: measured, projected, next limit	Briefing Q4
15	Concurrency card: lock order and scheduler lease	Guideline 4c
19	Pipeline numbers; checks corrected to what CI really runs	Briefing Q7; accuracy
20	Three-service timings from the 17 Oct run (51 min)	Accuracy
22	**New:** who and what can touch AWS (Organization guardrails, Identity Center, OIDC)	Briefing: minimum security controls
24	Three headline numbers	Takeaways
B1–B15	Backup slides with an index (BX); the Q&A answers point to them	Depth on demand

Corrections made while checking the v3 change document against the repository (18 Oct)

- **CodeQL does not run** (repository variable `ENABLE_CODEQL=false`). Static analysis is SonarCloud; slide 19 and the Q&A say so.
- **Service images are built and scanned, not booted one by one.** The backend application image is booted and its health endpoint checked on every pull request. Slide 19 says “build and scan every service image; boot the app and check its health”.
- **The “audit consumer” is a queue plus a filter:** two lines in `infra/swe5001/modules/messaging/main.tf` (PR #487); no service reads it yet. Slides 9 and 11 say “audit queue”.
- **Expiry notifications go to each active member** (`TeamExpiredV1.recipientUserIds` is filled from the team’s active members). Slide 16 and the script say “each member”.
- **The relay ceiling of about 50 events a second is calculated** (100 rows per 2-second tick, ADR 0007), not measured.
- **17 Oct run 39008208244:** build 3 min plus publish 2 min, create + deploy + smoke 32 min, release 1 min, destroy 13 min, 51 min end to end. The gaps between jobs are seconds, so “includes queueing and approval gaps” was removed.
- **No fix pull request exists** for the paused-scaling defect: the slide says “repaired by hand; no fix yet”.
- **Referral data exists** (`biz_share_invite`, `biz_share_credit`, `biz_purchase_intent.sharer_user_id`), so backup B11 can say it is stored.

Main deck: 24 slides (v2: 23). Segment timings are unchanged from Guideline V2.8; segment 5 fits five slides into its 6 minutes: 18 (65 s) + 19 (70 s) + 20 (95 s) + 21 (75 s) + 22 (55 s) = 360 s.

Run sheet and coverage check

Slide-by-slide run sheet

Clock	Slot	Slide	Title	Speaker	Item	Est.
16:15	15 s	1	BlocBuy	Lim	1a	14 s
16:15	40 s	2	Group buying only works if enough people join in time	Lim	1a	40 s
16:15	40 s	3	A two-sided platform built around one interaction: forming a team	Lim	1b	39 s
16:16	25 s	4	Scope: the full buying journey, on three services	Lim	1c	25 s
16:18	45 s	5	Status: what is verified on AWS, and what is still open	Lim	2a	43 s
16:18	45 s	6	On schedule, about 50 person-days across five people	Lim	2b	37 s
16:20	90 s	7	Why three services, not five	Eric	3a	72 s
16:21	80 s	8	Four decisions shape the logical design	Eric	3a	55 s
16:22	90 s	9	Logical view: one direct call, everything else by events	Eric	3b	68 s
16:24	75 s	10	Bounded contexts decide the services; events are the contract	Eric	3c	57 s
16:25	85 s	11	How producers and consumers use the platform, and what reuse costs	Eric	3c	60 s
16:29	85 s	12	Physical decisions: managed, disposable, low cost	Liu Jin	4a	63 s
16:30	85 s	13	Physical view: private services behind CloudFront, across two zones	Liu Jin	4b	53 s
16:31	45 s	14	Each service scales on its own signal, and how far it goes	Liu Jin	4b	43 s
16:32	45 s	15	Each service owns its data	Liu Jin	4c	45 s
16:33	95 s	16	Team expiry, end to end: one transaction in, one transaction out	Yorik	4d	80 s
16:34	65 s	17	Failures are expected, and handled	Yorik	4d	48 s
16:38	65 s	18	One main branch; production changes only by promotion	Pan	5a	38 s
16:39	70 s	19	Every pull request is checked before it can merge	Pan	5b	51 s
16:40	95 s	20	One promotion builds, deploys, tests and releases, then destroys it all	Pan	5b	67 s
16:41	75 s	21	What the real cloud taught us	Pan	5c	43 s
16:43	55 s	22	No standing keys: who and what can touch AWS	Pan	5b	47 s
16:45	15 s	23	Live demo	Eric	6a	10 s
16:53	Q&A	24	Thank you	Lim	6b	

Talk budget vs slots vs estimated speaking time: §1 120 s / 120 s / 118 s · §2 90 s / 90 s / 80 s · §3 420 s / 420 s / 312 s · §4 420 s / 420 s / 332 s · §5 360 s / 360 s / 246 s · §6 480 s / 15 s / 14 s (the §6 talk is the demo itself). The wording fills about 70% of each slot; the rest is for pointing at the slide and a natural pace.

Q&A windows (inside the 40 minutes): 16:17–16:18 · 16:19:30–16:20 · 16:27–16:29 · 16:36–16:38 · 16:44–16:45 · 16:53–16:55. Background actions: **16:05** Liu Jin lowers the commerce floor to 1 (A.7 (a)) · **16:23** Yorik captures the V3 preflight · **16:28** Yorik starts k6 V3 · **16:44:30** screen-share switch Lim → Eric.

Every guideline item has slides and a speaker

Item	Guideline requirement	Slides	Speaker
1a	Explain the context and the business problem that the solution as a whole is trying to achieve.	1, 2	Lim
1b	Highlight the platform-thinking aspects of your solution.	3	Lim
1c	Explain the scope of the project.	4	Lim
2a	Explain the current status of the project including outstanding issues.	5	Lim
2b	Describe the actual project milestones and the rough total effort expended by each team member.	6	Lim

3a	Highlight the key architectural decisions that influence the logical architecture.	7, 8	Eric
3b	Explain the logical components of the solution by showing a logical detail deployment diagram and other supporting views.	9 (UML logical deployment view)	Eric
3c	Where relevant, highlight the elements of Domain Driven Design that help to shape the design of the solution as a platform. In particular, describe how the platform is designed to interact with the producers and consumers.	10, 11	Eric
4a	Highlight the key architectural decisions that influence the physical architecture.	12	Liu Jin
4b	Describe the physical architecture that includes the network, nodes, technologies and cloud services that are used in the solution.	13, 14 (UML deployment)	Liu Jin
4c	Describe the persistence design of the solution.	15	Liu Jin
4d	Describe the detailed design of an architecturally representative user case or user story.	16, 17 (UML sequence)	Yorik
5a	Describe the strategy to manage the source code and artifacts.	18	Pan
5b	Describe the DevOps pipeline and procedures.	19, 20, 22	Pan
5c	Highlight any technical findings and issues encountered.	21	Pan
6a	Demonstrate some of the significant use cases of the solution.	23 + demo step 1	Eric
6b	Demonstrate how the solution deals with some of the important quality attributes e.g., scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).	24 + demo steps 2–3	Lim

Every briefing question has an answer on a slide

Briefing question or success criterion	Where
Q1 Pain points	2
Q2 Platform, common services, reusable frameworks	3, 9, 11; B5, B10
Q3 Benefit of building on the platform, demonstrated	11 (measured reuse)
Q4 Scale, and proof	14 (capacity statement), demo step 2; B12
Q5 Technical and non-technical benefits, demonstrated	12 (measured cost), 14, 19, demo
Q6 Value of cloud-native design, demonstrated	12 (US$0.49 measured), 13, 20
Q7 Which parts of the lifecycle are automated, and the effort saved	19 (5,252 runs), 20
Q8 Effort within the guideline	6
Success criterion: seed, producers, consumers	3
Success criterion: more seeds in future	3 (future seeds line)
Success criterion: analytics	B11 (Q&A)
Module: robust API access; minimum security controls	11, B10; 22

Cue words in the team backchannel (never the Zoom chat)

Message	Meaning	Sent by
`1 MIN`	Speaker has one minute of talk left in the segment	Lim
`TIME`	Stop talking; move to questions or the hand-over	Lim
`K6 GO` / `K6 UP`	Start the V3 load / load is running and healthy	Lim / Yorik
`SCALED n`	Commerce reached n running tasks (from the dashboard)	Eric
`CLIP n`	Live step n failed; Liu Jin is ready to share the recording for step n	Eric → Liu Jin

SHARE	Next person: start screen sharing now	Lim

1 Overview (2+1 minutes)

Clock 16:15:00 – 16:18:00 (2 min talk + 1 min Q&A; talk ends 16:17:00)
Owner Lim Jiechao
Slides 1, 2, 3, 4

Guideline requirement (V2.8)
1a — Explain the context and the business problem that the solution as a whole is trying to achieve.
1b — Highlight the platform-thinking aspects of your solution.
1c — Explain the scope of the project.

Slide 1 BlocBuy — 16:15:00–16:15:15 · 15 s

Speaker Lim · covers guideline item **1a**
v3 new colour design; content unchanged
Say *Good afternoon, we're Team 8. This is BlocBuy, a group-buying platform built to scale on AWS. We'll cover the problem, the design and how we ship it, then show it live.*

≈ 31 words · ≈ 14 s at 130 wpm · slot 15 s

Cues
- Lim opens swe5001-presentation-v3.html in Chrome, presses S for the notes window (keep it off the shared screen) and T there at 16:15:00. In Zoom, share only the deck window.
- Liu Jin watches ECS health on the dashboard, muted: commerce should be stable at 1/1 after the 16:05 floor change (run book A.7 (a)).

Hand-over *"So, what problem are we solving?"*

Slide 2 Group buying only works if enough people join in time — 16:15:15–16:15:55 · 40 s

Speaker Lim · covers guideline item **1a**
v3 unchanged
Say *Group buying means people buy together to get a lower price. But it only works if enough people join before the deadline. Most groups run this in chat apps and spreadsheets. Buyers can't see how close a deal is. Organisers collect the money and do refunds by hand. And small sellers have nowhere to post a team price. With BlocBuy, you pay to join a team, the price drops as more people join, and if the team doesn't fill in time, everyone gets their money back automatically.*

≈ 87 words · ≈ 40 s at 130 wpm · slot 40 s

Hand-over *"And we built it as a platform, not a shop."*

Slide 3 A two-sided platform built around one interaction: forming a team — 16:15:55–16:16:35 · 40 s

Speaker Lim · covers guideline item **1b**
v3 future seeds line added
Say *At the centre is one thing: forming a team at a team price. Sellers put up offers, and buyers browse and join them. When buyers start a team or share an invite, they bring in new people, so buyers help grow the platform too. Every new member makes the deal better for everyone; that's our network effect. We start it off with a few hand-picked offers. Sign-in, notifications and payments are shared, so a new kind of group buying, like event tickets, could reuse them.*

≈ 85 words · ≈ 39 s at 130 wpm · slot 40 s

Cues
- The future seeds are on the slide (briefing success criterion "support for more seeds in future"); no need to list them all aloud.

Hand-over *"Here's what we actually built."*

Slide 4 Scope: the full buying journey, on three services 16:16:35–16:17:00 · 25 s

Speaker Lim · covers guideline item **1c**

v3 evolution path added to out of scope

Say *We built the whole buying journey: sign in, browse, start or join a team, pay, and then the team either succeeds, or expires and everyone is refunded. It runs on three services. We left out multiple regions, real money and email. Moving payments into a service of its own is our planned next step.*

≈ 54 words · ≈ 25 s at 130 wpm · slot 25 s

Cues
- 16:17:00 — questions (1 minute). Lim repeats each question in one line and names who answers.

Hand-over *(after questions) "Now, where the project stands."*

Q&A (16:17–16:18)

Likely question	Suggested answer (say it in 30–45 s)	Who answers
What makes this a platform rather than one application?	Two services, identity and notification, are shared: any new kind of group buying can use them as they are. A new listener is just a new queue, and a new service starts from our template with login and error handling built in. Eric covers this in segment 3.	Lim → Eric
Why build group buying at all? Isn't this a solved problem?	Communities still organise it in chat groups. The hard parts are keeping track, collecting money and doing refunds, and the platform does all of that automatically.	Lim
What would a second business on the platform need?	Its own catalogue and rules, in a new service built from our template. Sign-in, notifications and payments are reused as they are. Examples are on slide 3.	Lim

2 Project Conduct (1.5+0.5 minutes)

Clock 16:18:00 – 16:20:00 (1.5 min talk + 0.5 min Q&A; talk ends 16:19:30)
Owner Lim Jiechao
Slides 5, 6

Guideline requirement (V2.8)
2a — Explain the current status of the project including outstanding issues.
2b — Describe the actual project milestones and the rough total effort expended by each team member.

Slide 5 Status: what is verified on AWS, and what is still open — 16:18:00–16:18:45 · 45 s

Speaker Lim · covers guideline item **2a**
v3 three columns: verified 17 Oct / tested 19 Oct / still open
Say *So where are we today? On 17 October we built the full three-service setup on AWS from nothing. All nine smoke tests passed, and teardown removed all 115 resources. On 19 October we ran the auto-scaling test on AWS, and commerce scaled itself from one task to three. The failure drills are tested locally only, as the middle column shows. Two things are still open. An interrupted start-up can leave auto-scaling switched off; for now we fix that by hand. And the database runs in one zone, a deliberate choice to save cost.*

≈ 93 words · ≈ 43 s at 130 wpm · slot 45 s

Cues
- Say only what docs/swe5001/evidence/ records; the assessors may probe status claims.
- The middle column says "not run on AWS" for every drill that was only tested locally; say it the same way.

Hand-over *"And how we got here."*

Slide 6 On schedule, about 50 person-days across five people — 16:18:45–16:19:30 · 45 s

Speaker Lim · covers guideline item **2b**
v3 actual milestone dates; role text expanded
Say *We stayed on the plan from our proposal. We handed it in on 4 October, deployed to AWS for the first time two days later, split into three services by the 14th, measured our load baseline on the 16th, made the first clean three-service release on the 17th, and ran our test session on the 19th. In total that's about 50 person-days, in line with what the course expects. Each person's share is based on the issues they actually delivered.*

≈ 80 words · ≈ 37 s at 130 wpm · slot 45 s
If short of time: If over time, name only three dates (4, 14 and 17 October).

Cues
- 16:19:30 — questions (30 seconds).

Hand-over *(after questions) "Eric will take you through the logical architecture."*

Watch out
Slide 5 shows the real state: only auto-scaling ran on AWS (tick); the drills say “not run on AWS”, with their local result where there is one. Never claim a result the evidence folder does not record.

Q&A (16:19:30–16:20)

Likely question	Suggested answer (say it in 30–45 s)	Who answers
How was the work split, and is the effort within the guideline?	About 50 person-days in total, which fits the course guideline: between 8.5 and 12.5 days per person. We added up the planned hours of the issues each person actually delivered, and the team agreed the final split.	Lim
What slipped?	Say it plainly from slide 5, then say how and when it will be closed. The full results table is backup B12.	Lim

3 Logical Architecture & Design (7+2 minutes)

Clock 16:20:00 – 16:29:00 (7 min talk + 2 min Q&A; talk ends 16:27:00)
Owner Wang Ziang (Eric)
Slides 7, 8, 9, 10, 11

Guideline requirement (V2.8)
3a — Highlight the key architectural decisions that influence the logical architecture.
3b — Explain the logical components of the solution by showing a logical detail deployment diagram and other supporting views.
3c — Where relevant, highlight the elements of Domain Driven Design that help to shape the design of the solution as a platform. In particular, describe how the platform is designed to interact with the producers and consumers.

Tip
Slide 7 is the most important slide of the talk: it is the one decision assessors will remember. Rehearse it until it fits in 90 seconds without rushing.

Slide 7 Why three services, not five — 16:20:00–16:21:30 · 90 s

Speaker Eric · covers guideline item **3a**
v3 decision tag AD-02 added
Say *I'm Eric, and I'll start with our biggest decision: where to split the system. The obvious way is one service per business area: catalogue, orders, payments, and so on. But look at what happens when a payment comes in from Stripe. In one database transaction, we confirm the payment, create the order, take the stock and add the buyer to the team. Four steps, one commit. If any step fails, all of it is undone, and that is what keeps a buyer's money safe. If we put those steps in separate services, we'd need complicated undo logic everywhere. We estimated five services at 50 to 55 person-days, our whole budget, so we said no. Our rule became simple: only split where no transaction crosses the line. Identity is used through one small interface, and nothing needs to call notifications. So we have two reusable platform services, and one core service that keeps the money flow together.*

≈ 156 words · ≈ 72 s at 130 wpm · slot 90 s

Cues
- Pause on the left box: this is the one decision assessors should remember.

Hand-over *"A few more decisions follow from that."*

Slide 8 Four decisions shape the logical design — 16:21:30–16:22:50 · 80 s

Speaker Eric · covers guideline item **3a**
v3 decision tags and cost line added
Say *Three more decisions follow from that, and each one solves a problem the split creates. First, services talk to each other through events, and we save each event in the same transaction as the change itself. So an event only goes out if the change really happened; we never announce a payment that was rolled back. Second, all services share one login session, so any service can check your role without asking identity every time. Third, if commerce can't reach identity, it refuses the purchase rather than guessing. A purchase is either fully done, or not done at all. The price we pay: notifications arrive about two seconds later, and purchases pause while identity is down. We've tested that locally.*

≈ 120 words · ≈ 55 s at 130 wpm · slot 80 s

Hand-over *"Here's how the pieces fit together."*

Slide 9 Logical view: one direct call, everything else by events — 16:22:50–16:24:20 · 90 s

Speaker Eric · covers guideline item **3b**
v3 UML logical deployment diagram replaces the box diagram
Say *This is our logical view. The two shared platform services are at the top, the business application is at the bottom, and the event system sits between them. Every request comes in through CloudFront and a load balancer, which sends it to the right service. There's only one direct call between services: commerce asks identity, "is this user still active?" That red arrow is the only place where one service waits for another, so it's the only place we had to plan for that failure. Everything else goes through events. Commerce saves each event in a small outbox table, a relay sends it to an SNS topic, and each listener gets its own queue. Today that's notifications and an audit queue. Adding the audit queue took two lines of configuration, and no change to commerce. Each service has its own database, and they all share one session store.*

≈ 148 words · ≈ 68 s at 130 wpm · slot 90 s

Cues
- Walk the diagram top to bottom: platform services, the red direct call, the event bus.
- The plainer v2 diagram is backup B13 if a simpler companion is needed in Q&A.

Hand-over *"These boundaries come from the domain model."*

Slide 10 Bounded contexts decide the services; events are the contract 16:24:20–16:25:35 · 75 s

Speaker Eric · covers guideline item **3c**
v3 unchanged (option A); the context map is backup B3
Say *We found these boundaries by following the life of a team: it's created, people join and pay, and it either succeeds, or expires and gets refunded. That gave us six areas of the business, what Domain-Driven Design calls bounded contexts. Four of them, catalogue, group buying, orders and payments, share transactions, so they live together in commerce. Identity and notification each stand alone. Inside commerce, these areas talk through internal events, like "team expired". When an event leaves the service, it becomes a public event with a version number, for example "team expired, version 1". It only carries IDs, never names or emails, so personal data stays in identity. And any new service can listen to these events without knowing how commerce works inside.*

≈ 124 words · ≈ 57 s at 130 wpm · slot 75 s

Cues
- If asked for the relationships between contexts, open backup B3 (context map).

Hand-over *"So how do the two sides use the platform?"*

Slide 11 How producers and consumers use the platform, and what reuse costs 16:25:35–16:27:00 · 85 s

Speaker Eric · covers guideline item **3c**
v3 measured reuse and the API contract added
Say *Here's how both sides use the platform. Producers are on the left: sellers and admins post listings, and buyers start and share teams. Consumers are on the right: they browse, join, track orders and get notified. Both sides use the same account. Whatever happens in commerce becomes an event, and a new service can simply subscribe to it. Reuse is cheap, and we can measure it. The audit queue took two lines of configuration and no change to commerce. A whole new service is one entry in our infrastructure template, which sets up everything it needs to run. All clients use one published API description, and our web app's client code is generated from it. And every new service starts with login, error handling, tracing and duplicate protection already built in.*

≈ 131 words · ≈ 60 s at 130 wpm · slot 85 s
If short of time: If over time, drop the last sentence (the strip at the bottom says it).

Cues
- 16:27:00 — questions (2 minutes).
- 16:28:00 — Yorik starts the k6 load test (muted) with the preflight captured at 16:23 (no more than five minutes old). Lim confirms in the backchannel.

Hand-over *(after questions) "Liu Jin will show how this runs on AWS."*

Q&A (16:27–16:29)

Likely question	Suggested answer (say it in 30–45 s)	Who answers
Isn't commerce still a monolith?	Yes, on purpose. One payment touches the payment, the order, the stock and the team in a single transaction. Splitting that up would need undo logic everywhere, about 50 to 55 person-days. Inside, commerce is still split into clear layers (backup B5), and moving payments out is our next step (backup B2).	Eric
Why not one service per business area?	Same reason: those areas share transactions. Point back to the four steps and the rejected five-service option on slide 7. The full context map is backup B3.	Eric
How does each service know a user's role?	When you sign in, identity writes your roles into a shared login session. The other services read that session, so they never have to ask identity on each request. A role change takes effect the next time you sign in.	Eric
Why do events have version numbers?	Because other services depend on them, like a public contract. A listener simply ignores versions it doesn't know, so we can add a version 2 without updating every service at the same time.	Eric
What did Domain-Driven Design actually change?	It gave us the candidate boundaries. Then we checked which of them share a transaction: those stayed together, and only the rest became separate services. It also made us separate internal events from public ones. Backup B4 shows the main objects and their states.	Eric
How would outside partners use the platform?	Through one published API description, with generated client code and a check on every pull request, plus the versioned events. Not built yet: an API gateway, keys per partner and rate limits. Backup B10.	Eric
Does anything read the audit queue yet?	Not yet. It's a queue that receives every team event, to show how cheap a new listener is. An audit or analytics service could read it later without any change to commerce.	Eric

4 Physical Architecture & Design (7+2 minutes)

Clock 16:29:00 – 16:38:00 (7 min talk + 2 min Q&A; talk ends 16:36:00)
Owner Liu Jin (4a–4c), Yu Qianshuo (Yorik) (4d)
Slides 12, 13, 14, 15, 16, 17

Guideline requirement (V2.8)
4a — Highlight the key architectural decisions that influence the physical architecture.
4b — Describe the physical architecture that includes the network, nodes, technologies and cloud services that are used in the solution.
4c — Describe the persistence design of the solution.
4d — Describe the detailed design of an architecturally representative user case or user story.

Tip
Liu Jin owns slides 12–15 (about 4.3 minutes); Yorik owns 16–17 (about 2.7 minutes). The hand-over line is on slide 15; Yorik unmutes during its last sentence. Slide 16 is the UML sequence with two click reveals; if it does not feel right at the 20 Oct rehearsal, present backup B15 (numbered steps) with the same words.

Slide 12 Physical decisions: managed, disposable, low cost — 16:29:00–16:30:25 · 85 s

Speaker Liu Jin · covers guideline item **4a**
v3 measured cost added
Say *I'm Liu Jin. On AWS, we run our containers on Fargate with ARM processors. There are no servers for us to manage, scaling is built in, and it's about 20 percent cheaper. Kubernetes would cost more just to run its control plane than all of our computing does. We create the whole environment when we need it, and destroy it afterwards. We use one database server with a separate database for each service, and a small NAT instance instead of AWS's NAT gateway: about 4 dollars a month instead of 43. There are no long-lived access keys anywhere. The trade-off is less redundancy, which is fine for a project like this. And this cost is measured, not guessed: seven full sessions from 15 to 17 October cost forty-nine cents in total for the database, cache and containers.*

≈ 137 words · ≈ 63 s at 130 wpm · slot 85 s

Hand-over *"Here's the resulting deployment."*

Slide 13 Physical view: private services behind CloudFront, across two zones — 16:30:25–16:31:50 · 85 s

Speaker Liu Jin · covers guideline item **4b**
v3 UML deployment diagram replaces the box diagram
Say *Here's how it's deployed: in Singapore, across two availability zones. Only the load balancer and the NAT sit in public subnets; the services, database and cache are all private. CloudFront is the only way in. It serves the web app and passes API calls to the load balancer, and the load balancer rejects anything without CloudFront's secret header. Container images are kept in ECR, secrets in Parameter Store, events go through SNS and SQS, and CloudWatch monitors everything. The blue boxes are permanent. Everything inside the red dashed box is rebuilt on every release. So security comes in layers: one public entry point, private services, no stored keys, and no secrets in the code.*

≈ 114 words · ≈ 53 s at 130 wpm · slot 85 s

Cues
- Point at the blue "permanent" boxes, then the red "rebuilt every release" box.
- Eric checks the dashboard backstage: from 1 task (the test floor), commerce should reach 2 tasks around 16:39 and 3 around 16:42.
- The plainer v2 diagram is backup B14.

Hand-over *"Each service scales on its own signal."*

Slide 14 Each service scales on its own signal, and how far it goes 16:31:50–16:32:35 · 45 s

Speaker Liu Jin · covers guideline item **4b**
v3 capacity statement added
Say *Commerce adds tasks as requests per task rise, from two to six; notification follows the number of waiting messages, from one to four. On 16 October one task handled 200 requests a second without reaching its limit. On 19 October, with all three services, commerce grew from one task to three and served about 455 requests a second, 95% of them in under 50 milliseconds. That's as much as our laptop could send. Six tasks should handle about 1,200 a second, but that's an estimate, and beyond it the single database is the limit.*

≈ 94 words · ≈ 43 s at 130 wpm · slot 45 s
If short of time: If over time, skip the chart; read only the capacity line.

Cues
- Never present the projection as a measurement. 455 req/s was the laptop's limit, not the service's: if asked, say so.

Hand-over *"Now the data."*

Slide 15 Each service owns its data 16:32:35–16:33:20 · 45 s

Speaker Liu Jin · covers guideline item **4c**
v3 concurrency card added
Say *Each service owns its own data. They share one database server, but each has its own database, and permissions stop a service from reading anyone else's tables. The cache only holds login sessions, and we never store card details. One tricky case is two buyers grabbing the last seat at the same moment. We always lock records in the same order, team first, then the purchase, then the payment, so exactly one buyer wins. And scheduled jobs take a lock in the database, so only one copy runs. Yorik will now walk through one use case in detail.*

≈ 98 words · ≈ 45 s at 130 wpm · slot 45 s

Hand-over *"Yorik, over to you."*

Slide 16 Team expiry, end to end: one transaction in, one transaction out 16:33:20–16:34:55 · 95 s

Speaker Yorik · covers guideline item **4d**
v3 UML sequence diagram with numbered steps
Say *I'm Yorik. This is what happens when a team runs out of time. Every minute, a scheduled job looks for expired teams, and a database lock makes sure only one copy of it runs. In one transaction, it marks the team as expired, cancels the orders, records the refunds that are owed, and saves a "team expired" event. Either all of that happens, or none of it does. [click] Once that's committed, the relay sends the event to SNS, which puts it on the notification queue. The notification service picks it up, checks the event ID so a duplicate is ignored, and creates a notification for each member, again in one transaction. Only then is the message deleted from the queue. [click] Refunds run on their own. Every thirty seconds, a refund worker takes the next refund and sends it to Stripe with a unique key, so even if it retries, nobody is refunded twice. So: one transaction in commerce, one in notification, a queue in between, and refunds that are safe to retry.*

≈ 174 words · ≈ 80 s at 130 wpm · slot 95 s

Cues
- Two clicks: the slide opens on the expiry transaction (step 1); click 1 shows relay → SNS → SQS → notification (steps 2–8); click 2 shows the refund worker loop.
- Fallback if the UML version does not feel right in rehearsal: backup B15 (numbered steps) with the same words.

Hand-over *"And when things go wrong?"*

Slide 17 Failures are expected, and handled 16:34:55–16:36:00 · 65 s

Speaker Yorik · covers guideline item **4d**
v3 unchanged
Say *A queue can deliver the same message more than once, so we designed for that. If a message arrives twice, the second copy is ignored, because we've already recorded its ID. If a message keeps failing, after five tries it's moved aside into a dead-letter queue and an alarm goes off, while everything else keeps flowing. We tested this locally: a broken message was tried five times, then moved aside. And if sending an event fails, it simply stays in the outbox and is tried again. So no event is lost, nobody gets the same notification twice, and one bad message can't block the rest.*

≈ 105 words · ≈ 48 s at 130 wpm · slot 65 s

Cues
- 16:36:00 — questions (2 minutes). Liu Jin answers infrastructure and data; Yorik answers events and failures.

Hand-over *(after questions) "Pan will show how we build and ship it."*

Q&A (16:36–16:38)

Likely question	Suggested answer (say it in 30–45 s)	Who answers
What breaks first at higher scale?	The single database server, then the relay that sends events, at about 50 events a second. That number is calculated, not measured. The fixes: give busy databases their own server or read copies, and run several relays side by side.	Liu Jin
Aren't one database and one cache single points of failure?	Yes, and that was a cost choice for this project. The services already run across two zones. For production we'd add a standby database and a cache replica (backup B2, AISS-01).	Liu Jin
Why Fargate and not Kubernetes?	Kubernetes on AWS charges about 73 dollars a month just for its control plane, more than all our computing. Fargate scales on its own and has no servers to manage, and three services don't need Kubernetes. Backup B1b.	Liu Jin
How are the internal endpoints protected?	The load balancer doesn't route them, only our own services can reach them, and they're not in the public API; a test checks that. There's no service token yet. We accepted that limit (AISS-04).	Liu Jin
Where does the 12,000 requests per minute target come from?	Our 16 October load test: one task handled 200 requests a second with fast responses and no errors, and it still had room. 200 a second is 12,000 a minute. On 19 October, with all three services, that target scaled commerce from one task to three.	Liu Jin
Slide 14 says two to six, so why does the test start at one?	Two is the floor we normally run with: one task in each availability zone. The scaling test is designed to start from one task, because our per-task figure, 200 requests a second, was measured on one task. Starting from one gives a clean baseline, and the test tool checks it before it sends any traffic. So we lower the floor to one for the test and set it back to two afterwards. (Do not claim the two tasks were verified in two zones after 19 October: that was not checked.)	Liu Jin
How do you make sure events are never lost or doubled?	Each event is saved in the same transaction as the change, so it can't be lost. A queue may deliver it twice, so each listener records the event ID together with its own change, and a repeat does nothing.	Yorik
What happens when identity is down?	Browsing keeps working. Purchases and admin changes fail safely with a server error. For a Stripe payment, everything is undone and Stripe tries again later. We show this live. The limit: checkout needs identity to be up (AISS-03).	Yorik
Why SNS and SQS, and not Kafka or RabbitMQ?	They're fully managed and you pay only for what you use, with nothing to run. SNS copies each event to every queue that wants it,	Yorik

	and the queue length drives scaling and gives us dead-letter queues. Kafka would be far too much for our volume.	
Can you show the payment path instead of expiry?	Yes, backup B6: the signature is checked first, then one transaction checks for duplicates, locks the records, adds the member, saves the order, takes the stock and saves the event.	Yorik

5 DevOps and Development Lifecycle (6+1 minutes)

Clock 16:38:00 – 16:45:00 (6 min talk + 1 min Q&A; talk ends 16:44:00)
Owner Panpakorn Siripanich (Pan)
Slides 18, 19, 20, 21, 22

Guideline requirement (V2.8)
5a — Describe the strategy to manage the source code and artifacts.
5b — Describe the DevOps pipeline and procedures.
5c — Highlight any technical findings and issues encountered.

Tip
Segment 5 now has five slides in the same 6 minutes. The hand-over to the demo moved from slide 21 to slide 22.

Slide 18 One main branch; production changes only by promotion 16:38:00–16:39:05 · 65 s

Speaker Pan · covers guideline item **5a**
v3 slot 75 → 65 s
Say *I'm Pan. All work happens on short-lived branches that merge into our main branch, dev, after a review and automated checks. Production lives on a separate branch, prod. The only way to change it is a promotion pull request from dev; nobody can push to it directly. For build outputs, we store each container image by its unique digest rather than by a name tag, keep a record of which images are live, and keep one API description file in the repository.*

≈ 82 words · ≈ 38 s at 130 wpm · slot 65 s
Hand-over *"Every change is checked before it merges."*

Slide 19 Every pull request is checked before it can merge 16:39:05–16:40:15 · 70 s

Speaker Pan · covers guideline item **5b**
v3 pipeline numbers added; checks corrected to what CI runs
Say *Every pull request goes through four groups of checks. First, build and test: unit tests, an API check, and at least 80 percent test coverage on new code. Second, containers: we build and scan every service image, and start the app to make sure it's healthy. Third, security: we scan for leaked secrets, check our libraries and images for known weaknesses, and run code analysis in SonarCloud. Fourth, infrastructure: we check and test our Terraform code and run policy checks. Over the past month that came to more than five thousand runs, about 95 percent passing, usually in under nine minutes. If any check fails, the change can't be merged.*

≈ 110 words · ≈ 51 s at 130 wpm · slot 70 s
If short of time: If over time, one sentence for all four groups.
Hand-over *"And this is how a change reaches production."*

Slide 20 One promotion builds, deploys, tests and releases, then destroys it all 16:40:15–16:41:50 · 95 s

Speaker Pan · covers guideline item **5b**
v3 three-service timings from the 17 Oct run
Say *This is the key part. The only thing that deploys to production is merging a promotion pull request. The pipeline then builds the images, creates the whole AWS environment from scratch, deploys the services and runs smoke tests through CloudFront. If a release fails its health checks, it rolls itself back. Then the pipeline marks those images as the new release,*

destroys everything, and checks that nothing was left behind. That last step always runs, even if something failed earlier. Because every release starts from nothing, every release is also a disaster-recovery test. Our first run on 6 October, with one service, took 28 minutes. The three-service run on 17 October took 51. About half an hour of that is creating the environment, and fifteen minutes of it is just waiting for new SNS subscriptions to start working. Which brings me to what we learned.

≈ 145 words · ≈ 67 s at 130 wpm · slot 95 s

Cues
- Timings from run 39008208244 (17 Oct): build 3 min + publish 2 min, create + deploy + smoke 32 min, release 1 min, destroy 13 min.

Hand-over *"Three lessons we learned on the way."*

Slide 21 What the real cloud taught us — 16:41:50–16:43:05 · 75 s

Speaker Pan · covers guideline item **5c**
v3 slot 95 → 75 s; hand-over moved to slide 22
Say *The real cloud taught us three things. First, our infrastructure tests passed with mock-ups, but the first real deployment failed because of missing permissions. Mocks only test what you tell them, so now we test permissions for real. Second, a new SNS subscription can take up to fifteen minutes to start delivering. Nothing shows an error; the messages just disappear. So our deploy now waits until delivery is confirmed. Third, a start-up that was interrupted and retried said it had succeeded, but left auto-scaling switched off. Deployed doesn't mean scaling, so now we check.*

≈ 94 words · ≈ 43 s at 130 wpm · slot 75 s

Cues
- The full findings list is backup B8.

Hand-over *"And who and what can touch AWS."*

Slide 22 No standing keys: who and what can touch AWS — 16:43:05–16:44:00 · 55 s

Speaker Pan · covers guideline item **5b**
v3 NEW (briefing: minimally required security controls)
Say *Finally, who and what can touch our AWS account. Security starts above the account itself. It sits in an AWS Organization whose rules keep us in Singapore and block expensive services, and those rules apply even to our own admins. People sign in through Identity Center, with sessions that end after an hour. The pipeline gets short-lived access from GitHub, with separate roles to plan, deploy and apply, so no AWS keys are stored in GitHub. There's one exception, and we're open about it: two IAM users for database access, still under the same rules. Eric will now show the system live.*

≈ 102 words · ≈ 47 s at 130 wpm · slot 55 s

Cues
- 16:44:00 — questions (1 minute).
- 16:44:30 — Lim stops sharing; Eric starts sharing the demo window.

Hand-over *(after questions) "Eric, over to you."*

Watch out
At 16:44:30 Lim stops sharing and Eric starts sharing the demo window, even if a question is still being answered. The answer continues over Eric's screen.

Q&A (16:44–16:45)

Likely question	Suggested answer (say it in 30–45 s)	Who answers
How do you stop one change from breaking a service?	Every pull request builds and scans every service image and starts the app to check it's healthy. On top of that, 18 automated architecture rules stop code from crossing service boundaries (backup B5).	Pan

Why destroy the environment after every release?	Cost, and proof. Running only when needed costs about 20 cents an hour instead of about 125 dollars a month; seven real sessions cost 49 cents. And because every release builds everything from nothing, we know we can rebuild at any time. Backup B9.	Pan
What security checks are in the pipeline?	Scans for leaked secrets, for known weaknesses in our libraries and images, and for risky infrastructure settings, plus code analysis in SonarCloud. Deploy roles only work for the prod branch, and secrets live in Parameter Store, never in code. The account-level rules are on slide 22.	Pan
How do you roll back?	If a new version fails its health checks, ECS rolls back to the last good one by itself. We also keep a record of the last good images, so we can always rebuild from them.	Pan
Why are there still IAM users?	Two users for database access, with one active key, shown openly on slide 22. They're still under the organisation's rules. Everyone and everything else signs in through Identity Center or short-lived GitHub access.	Pan → Lim

6 Demo (8+2 minutes)

Clock 16:45:00 – 16:55:00 (8 min talk + 2 min Q&A; talk ends 16:53:00)
Owner Eric and Yorik (drive), Pan (pipeline)
Slides 23

> **Guideline requirement (V2.8)**
> **6a** — Demonstrate some of the significant use cases of the solution.
> **6b** — Demonstrate how the solution deals with some of the important quality attributes e.g., scalability (such as demonstrating auto-scaling), performance (show using some scripted performance testing), maintainability (show/run the DevOps pipeline).

How the demo meets 6a and 6b

Step	Shows	Quality attribute (guideline 6b wording)	Evidence on screen
1	Buy as a customer, across three services	Functionality	All four views load; at least one notification is listed.
2	Auto-scaling under a load test	Scalability, performance	Commerce running tasks above 1 (3 expected, as on 19 Oct); server p95 back under 100 ms once the new tasks run; almost no errors. k6 on the laptop may show about 0.5 s and dropped requests: that is the laptop's limit, not the service.
3	The pipeline that deployed it	Maintainability	Checks green; run green including destroy; artifact present.
4	Close the demo	Reliability (tested locally)	Nothing to operate. 16:51:30–16:53 is spare: questions on the demo.

Scalability and performance are shown live through the scripted k6 test in step 2, and maintainability through the pipeline in step 3. Three drills were cut from the live demo on 19 October because they were not rehearsed on AWS: stopping a server, the identity outage and the event backlog with the dead-letter queue. Slide 23 says so in one line, and slide 5 and backup B12 show them as tested locally. The demo now ends at about 16:51:30; the spare time goes to questions.

Principle: start everything slow before the demo

The demo only **shows results**; nothing is changed on the environment live. The k6 V3 run starts at 16:28 and follows the reviewed ramp: 20% → 100% of the 200 RPS baseline over 5 minutes (16:28–16:33), up to 300% = 600 RPS over 10 minutes (16:33–16:43), hold 10 minutes (16:43–16:53), ramp down 5 minutes. On 19 October the same ramp from a laptop topped out at about 455 RPS (k6 hit its 400-user limit), and commerce scaled 1 → 2 at 15:55 and 2 → 3 at 15:58, about 11 and 14 minutes into the ramp. The talk starts from the same point as 19 October: commerce at 1 task, because the k6 V3 preflight only accepts a stable one-task deployment with a scaling floor of 1 (A.7). So expect the same timing: 2 tasks around 16:39 and 3 around 16:42, before step 2 at 16:46:45. It will stay at 3: at about 455 RPS each task receives about 9,200 requests a minute, below the 12,000 target, and the laptop cannot push enough for a fourth. k6 on the laptop will show about 0.5 s and dropped requests while the server answers in under 50 ms; read the server numbers off the dashboard, not k6.

Demo preconditions (checked at 15:30 by Liu Jin and Eric)

- Environment up since 19 Oct (kept running until the talk), health `UP`, smoke green; commerce 2/2 (its standing floor), identity 1/1, notification 1/1 running.
- Demo admin granted and signed in again; startable listings seeded (A.5).
- Scaling targets **not suspended**: commerce 2–6 as deployed, notification 1–4 (read back; see finding 3 on slide 21). At about 16:05 commerce is set to 1–6 with desired count 1 for the load test, and must be stable at 1/1 before the preflight (A.7); notification stays 1–4. Neither may be suspended.
- Notifications JSON for the demo buyer shows at least one entry (from the smoke run or a seeded expiry).

- Load machine: Docker running, k6 image pulled, `BASE_URL`, digest and run label exported (A.7). The V3 preflight is taken at 16:23, after the floor change: run before 16:05 it fails by design, because it requires one running commerce task.
- Fallback clips (recorded before the talk) open in a player on Liu Jin's machine, one per step, tested with Zoom "share sound" off. No recordings yet: any step that fails live uses Plan C (§7) until they are recorded.

Screen layout (Eric's demo window)

One browser window, tabs left to right, plus one terminal. Share the window, not the desktop; notifications off; browser zoom 125%.

Tab	Content	Prepared state
1	HTML deck opened at slide 23 (`swe5001-presentation-v3.html#23`)	Also used for slide 24 at the end; the backups are one key away (**B**)
2	SPA `https://d3lvq02o2e7tcn.cloudfront.net`	Product list loaded; signed in as the demo admin (Google)
3	Admin product list in the SPA	Admin listing visible
4	`/api/v1/notifications` for the demo buyer	JSON list with at least one notification
5	CloudWatch dashboard `blocbuy-swe5001`	1-hour window, auto-refresh 10 s; Edge, Services, Messaging rows visible
6	GitHub: promotion pull request #534 (17 Oct) (`dev` → `prod`) and its `swe5001-deploy` run	Checks tab and run graph open; never the Merge button
T	Terminal mirroring k6 on the load machine	Live summary: request rate, p95, error rate

Opening (slide 23)

Slide 23 Live demo 16:45:00–16:45:15 · 15 s

Speaker Eric · covers guideline item **6a**
v3 footer wording depends on what is live
Say *Over the next few minutes: the buying journey, auto-scaling under a load test running since 16:28, and the pipeline that deployed it.*

≈ 22 words · ≈ 10 s at 130 wpm · slot 15 s

Cues
- Eric's tab 1 shows this slide (swe5001-presentation-v3.html#23). Follow the demo run book step by step.

Hand-over *"First, the use cases."*

The slide's footer reads "live on AWS; any recorded step is marked" until `demoLive` is set to `true` in the results block; set it only if every step will be live.

Step by step

Step 1 Buy as a customer, across three services 16:45:15 – 16:46:45

Proves Functionality · guideline **6a** · Eric
Operate
1. Tab 2: product list with team prices and countdowns (commerce).
2. Point at the signed-in user in the header (identity, Google sign-in).
3. Tab 3: admin product list (commerce reads the role from the shared session).
4. Tab 4: refresh the buyer's notifications (notification service).

Say *This is what a buyer sees: team prices and live countdowns, served by commerce. I'm signed in with Google through the identity service. This admin page works because commerce reads my role from the shared login session, without calling identity. And these notifications were created by the notification service, from events it received. Three services, one login.*
Expect All four views load; at least one notification is listed.
Fallback Screenshots taken before the talk (fallback/1-use-cases). If the live view is not as expected within 20 seconds, Eric posts `CLIP 1` and says: *"Here is the same step from our recording."* With no recording for this step, use Plan C (§7).

Step 2 Auto-scaling under a load test 16:46:45 – 16:49:15

Proves Scalability, performance · guideline **6b** · Eric (Yorik mirrors k6)

Operate
1. Tab 5: CloudWatch dashboard, Services row: commerce running tasks.
2. Edge row: request count and p95 response time.
3. Terminal: k6 live summary (requests per second, p95, error rate).

Say *Since 16:28, a test script has been sending browsing traffic, ramping up towards three times our measured baseline. Commerce started with one task, the floor we use for this test, and has scaled up to what you see here. Response times went up while the new tasks were starting, then came back down, and almost no requests failed. On 19 October the same test took commerce from one task to three, at about 455 requests a second. (Read the task count and server response time off the screen.)*

Expect Commerce running tasks above 1 (3 expected, as on 19 Oct); server p95 back under 100 ms once the new tasks run; almost no errors. k6 on the laptop may show about 0.5 s and dropped requests: that is the laptop's limit, not the service.

Fallback The 19 Oct dashboard graphs (evidence folder 20261019-v3-autoscaling): task count 1 → 3 and server response time. If the live view is not as expected within 20 seconds, Eric posts `CLIP 2` and says: *"Here is the same step from our recording."* With no recording for this step, use Plan C (§7).

Step 3 The pipeline that deployed it 16:49:15 – 16:50:45

Proves Maintainability · guideline **6b** · Pan narrates; Eric clicks

Operate
1. Tab 6: promotion pull request #534 (17 Oct): required checks and approval.
2. Open its deploy run: build → create → release → destroy → evidence, with durations.
3. Open the evidence artifact list. Never click Merge.

Say *This is the promotion that put today's code on AWS: three required checks and one approval. The run built the images, created the environment from scratch, tested it, released it and destroyed it, and here's the evidence it saved. Today's environment was built from exactly these images. (Read the total run time off the screen.)*

Expect Checks green; run green including destroy; artifact present.

Fallback Screenshots of the pull request and run (fallback/3-pipeline). If the live view is not as expected within 20 seconds, Eric posts `CLIP 3` and says: *"Here is the same step from our recording."* With no recording for this step, use Plan C (§7).

Step 4 Close the demo 16:50:45 – 16:51:30

Proves Reliability (tested locally) · guideline **6b** · Eric, then Lim

Operate
1. Tab 5: commerce task count (still at its peak; the load runs until about 16:58).
2. Tab 1: thank-you slide (slide 24) at 16:53, or open backup B12 if asked about failures.

Say *When the load test ends, commerce will scale back down by itself. What we didn't show live is failure handling: stopping a server, an identity outage and a broken message. We tested those locally, and the results are in our backup slides. That's our demo.*

Expect Nothing to operate. 16:51:30–16:53 is spare: questions on the demo.

Fallback —

Watch out

Never live: clicking Merge on any PR into `prod` (a promotion destroys this environment); opening Parameter Store values, Terraform state or plan output; scaling commerce or notification by hand (autoscaling owns them); running `env-up.sh` while k6 is running; stopping tasks or services (the drills are not part of this demo).

Tip

Narrate results, not clicks: "task count went from two to three", not "now I'm clicking on the services row". No troubleshooting on air.

Closing slide

Slide 24 Thank you 16:53:00 · Q&A

Speaker Lim · covers guideline item **6b**
v3 three headline numbers added
Say *Thank you. We're happy to take your questions.*

≈ 8 words · ≈ 4 s at 130 wpm

Cues
- 16:55:00 — end. Lim thanks the assessors. Liu Jin tears the environment down only after the meeting ends.
- Backup slides follow (press → or X for the index). Open one only when a question needs it.

Q&A (16:53–16:55)

Likely question	Suggested answer (say it in 30–45 s)	Who answers
How did you choose the scaling target?	From the 16 October load test: responses stayed fast and errors stayed under one percent all the way to 200 requests a second per task. That gives 12,000 a minute. On 19 October, with all three services, that target scaled commerce from one task to three.	Eric
Does adding tasks scale in a straight line?	Not proven yet. On 19 October one task topped out near 280 requests a second and three tasks served about 455, but the laptop sending the load hit its own limit first, so we can't claim a straight line. Backup B12.	Eric
What analytics does the platform have?	Operational ones are in use: the dashboard, the alarms, and the load results that set our scaling target. Business ones, like how fast teams fill or how many referrals come from invites, are possible from data we already store, but not built yet. The next step is a reporting service that listens to our events. Backup B11.	Lim
Can you show … again?	The demo screen stays up. Eric drives, Pan shows the pipeline. Failure drills are not shown live: open backup B12.	Lim routes
Questions left over from earlier segments	That segment's owner answers in 30–45 seconds, and opens the matching backup slide if it helps.	Lim routes

7 Contingencies and time recovery

If something fails

Situation	Detected by	Response and words	Who
Environment down or unhealthy on the day	Morning check	Run `up` on `prod` (about 32 min), deadline 12:00; if it fails, `down` and retry once. If it fails again, present the demo from the recordings and the 19 Oct graphs and say so.	Liu Jin
Presenter's audio or connection drops	Silence > 5 s	Backup continues from the slide card: Lim ↔ Pan, Eric ↔ Yorik, Liu Jin ↔ Yorik.	Backup
Lim cannot share	Black screen	Eric shares the HTML deck from tab 1 for segments 1–5 (PDF as a last resort).	Eric
k6 does not start at 16:28	No `K6 UP` by 16:30	Yorik retries once before 16:31, with a fresh preflight (A.7 (b)) because the 16:23 one is then too old; otherwise step 2 shows the 19 Oct dashboard graphs (evidence folder `20261019-v3-autoscaling`).	Yorik
Commerce floor change not stable by 16:20	A.7 (a) wait has not returned	No preflight, so no live load: step 2 shows the 19 Oct dashboard graphs (task count 1 → 3). Do not start k6 against a two-task deployment; the preflight rejects it.	Liu Jin → Yorik
Commerce below 3 tasks at 16:46	Dashboard at 1 or 2 tasks (expected: 2 by about 16:39, 3 by about 16:42)	Show what is visible honestly ("it has scaled from one to two so far; the alarm needs three minutes of data"), then the 19 Oct dashboard graphs (task count 1 → 3).	Eric
GitHub slow or logged out	Tab 6	Show the pipeline screenshots.	Pan
Plan C: a step fails live and there is no recording for it	—	Show the local integration-test result for that step and say: "verified locally, not yet on AWS." Never present a local result as an AWS one.	Eric
Assessor asks for something not prepared	—	Offer to show it after the talk or in the report; do not improvise on the live environment. If a backup slide covers it, open it (**B**, then the row).	Lim

If the clock slips

Recover time in this order; never cut a guideline item, only depth.

Segment	Cut first	Saves
1	Slide 3: skip the hand-picked offers sentence; slide 4: skip the last sentence (next step)	15 s
2	Slide 6: name only three milestone dates	15 s
3	Slide 8: name the decisions, skip the cost sentence; slide 11: drop the last sentence (the strip says it)	35 s
4	Slide 12: skip the Kubernetes sentence; slide 14: skip the chart; slide 15: skip the cache and card sentence	45 s
5	Slide 19: one sentence for all four groups; slide 21: two findings instead of three; slide 22: skip the disclosed exception (it is on the slide)	50 s
6	Step 1: show the product list and notifications only; the demo already ends 1.5 min early	30 s

8 Timeline before, on and after the day

Before the day

Date	Do	Owner
Sun 18 Oct	Each owner reviews their v3 slides and this script and confirms their effort figure on slide 6; send changes to Lim.	All; Lim collects
Mon 19 Oct	Done: V3 auto-scaling ran on AWS (evidence in PR #555); the other drills stay local and were cut from the live demo. Decision: keep the environment up until the talk; no promotion to `prod` before it (a promotion destroys the environment).	All; Liu Jin operates
Mon 19 Oct evening	Done: results block filled, `final: true`, deck, PDF and this script rebuilt with no gaps left.	Lim, with segment owners
Tue 20 Oct	**Record the fallback clips** for demo steps 1–3 on the running environment (screen + k6 terminal, 1080p, no secrets visible) into `fallback/1-use-cases` … `fallback/3-pipeline`. **Full rehearsal** in Zoom, timed against the run sheet, with both share switches and the real 16:28 cue on the clock. Confirm slide 16 (UML) or its fallback B15. Every speaker answers their Q&A rows in ≤ 45 s.	All; Lim times

Presentation day (Wed 21 Oct)

Time (SGT)	Do	Owner
≤ 10:30	A.1: nothing queued that could destroy the session. The environment has stayed up since 19 Oct: check it with A.2's health and scaling commands. Run `up` on `prod` only if it is down (about 32 min).	Liu Jin
≤ 12:00	SPA loads, health `UP`, smoke green; scaling targets read back as not suspended. Sign in with the demo Google account, A.5 grant-admin, sign out and in, seed listings. If `up` failed: cause → `down` → retry once.	Liu Jin
13:30	Tech check in a test Zoom: mic, camera, share window, notifications off, wired network if possible. Load machine ready (A.7 exports, preflight).	All
15:30	Open the demo tabs; run through the §6 preconditions.	Liu Jin, Eric
16:00	Join the meeting as "Team 8 – Name"; backchannel open; fallback clips loaded.	All
16:05	A.7 (a): lower the commerce floor to 1–6 and desired count 1; wait for a stable 1/1 deployment. Needs Identity Center administrator rights.	Liu Jin
16:15	Start; follow the run sheet.	Lim
16:23	A.7 (b): capture the V3 preflight, no earlier than five minutes before the load.	Yorik
16:28	`K6 GO` → Yorik starts V3 (A.7).	Yorik
16:44:30	Share switch Lim → Eric.	Lim, Eric
16:55	End; thank the assessors.	Lim
≤ 17:30	Export the dashboard screenshot, k6 summary and ECS events to the evidence folder. A.9: `down` and a clean `status`. Write every assessor remark into #461.	Liu Jin, Lim

After the presentation

- Teardown verified (A.9); record both run links in the day log. Save evidence **before** teardown: log groups die with `runtime`.
- Every assessor remark goes into #461 the same evening; each becomes a change or a justified non-change in the report (due 16 Nov).
- Tag `swe5001-demo-20261021` on the `prod` commit that was shown (never a `v*` tag).

9 Checks and open items before the rehearsal

Checked against the repository (closed)

#	Item from the v3 change document	Finding and source	Slide
1	Audit consumer: lines and files	A queue plus a filter: 2 lines in `infra/swe5001/modules/messaging/main.tf` (PR #487); no service reads it yet	9, 11
2	Expiry notification: owner or each member	Each active member (`IntegrationOutboxWriter.onTeamExpired` → `findActiveMemberIdsByDealId`)	16, B15
3	Paused-scaling defect: fix PR open?	No fix PR; repaired by hand	5, B8
4	Boot check and CodeQL in CI	Service images built and scanned (`swe5001-service-images.yml`); backend app booted with a health check (`ci-backend.yml`); CodeQL off (`ENABLE_CODEQL=false`)	19
5	Relay ceiling of about 50 events/s	Calculated: 100 rows per 2-second tick (ADR 0007), not measured	14, Q&A
6	17 Oct run 39008208244 durations	Build 3 + publish 2, up 32, release 1, down 13 min; 51 min end to end; gaps of seconds	20
9	Q&A bank: single-zone data is AISS-01	Corrected in this script	Q&A
10	Referral data for analytics	Stored: `biz_share_invite`, `biz_share_credit`, `biz_purchase_intent.sharer_user_id`	B11
11, 12	Slide 22, B10, B11 graphics; recoloured UML diagrams	Built in v3 (diagrams taken from the team's v3 PDF)	9, 13, 16, 22, B3–B6
14	Slide 10 option A or B	Option A; the context map is backup B3	10
7	Effort split (slide 6)	Agreed by all five on 19 Oct	6
16	Demo steps 3–5 (stop a server, identity outage, event backlog)	Cut on 19 Oct: not rehearsed on AWS; slide 23 says they were tested locally	23
8	19 Oct results	V3 auto-scaling ran on AWS: commerce 1 → 3 tasks, about 455 RPS at server p95 under 50 ms; not V3 acceptance (generator drops). v4, v5, v7, identity, e2e shown as "not run on AWS". Evidence: `docs/swe5001/evidence/20261019-v3-autoscaling/` (PR #555)	5, 14, 24, B12
15	Rebuild after 19 Oct	HTML, slide PDF and this script rebuilt with `final: true`	all

Still open

#	Item	Owner	Blocks slide
—	Record the fallback clips for demo steps 1–3, then set `recorded` in the results block and rebuild	Liu Jin	23
—	Review and merge the V3 evidence pull request #555	Eric, Yorik	—
13	Yorik rehearses the UML expiry sequence; fallback is B15	Yorik	16
—	Alarm e-mail confirmation before the final release (#515)	Liu Jin	5
—	Demo footer: set `demoLive` to `true` only if every step will be live	Eric	23

Effort per member

Estimated from the planned hours of the issues each person delivered, credited to whoever actually did the work, and adjusted by the team. Total ≈ 50 person-days; each member confirms their figure.

Member	Person-days	Main area
Liu Jin	12.5	AWS stacks, pipeline, service split
Panpakorn (Pan)	11.5	Messaging, scaling, login, load tests
Yu Qianshuo (Yorik)	9	Event outbox and relay, failure drills
Lim Jiechao	8.5	Lead; AWS accounts and guardrails; architecture study; presentation
Wang Ziang (Eric)	8.5	Notification consumer, dead-letter redrive, demo lead

A Appendix — operator commands

Run from an operator environment with the reviewed checkout (see the environment session runbook). Never paste outputs containing parameter values into chat or onto the shared screen.

A.1 Nothing queued that could destroy the session

```
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow swe5001-env.yml --limit 5
gh run list --repo PPsyrius/iss-groupbuy-team8 --workflow swe5001-deploy.yml --limit 3
```

A.2 Bring the environment up (release digests) and check health

```
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=up
curl -s https://d3lvq02o2e7tcn.cloudfront.net/api/actuator/health
aws application-autoscaling describe-scalable-targets --service-namespace ecs \
  --query 'ScalableTargets[].
{id:ResourceId,min:MinCapacity,max:MaxCapacity,suspended:SuspendedState}'
```

A.3 V5 self-healing — stop one commerce task under load (not used in the demo)

```
aws ecs list-tasks --cluster blocbuy-swe5001 --service-name commerce-svc
aws ecs stop-task --cluster blocbuy-swe5001 --task <task-arn> --reason "V5 drill"
aws ecs describe-services --cluster blocbuy-swe5001 --services commerce-svc \
  --query 'services[0].events[:5]'
```

A.4 Identity outage drill — stop and restore (not used in the demo)

```
aws ecs update-service --cluster blocbuy-swe5001 --service identity-svc --desired-count 0
aws ecs update-service --cluster blocbuy-swe5001 --service identity-svc --desired-count 1
```

A.5 Demo admin and startable listings

```
infra/swe5001/scripts/grant-admin.sh '<demo google email>'
BLOCBUY_BASE_URL=https://d3lvq02o2e7tcn.cloudfront.net/api/v1 \
  BLOCBUY_TOKEN=<cookie value> scripts/seed-dev-demo-listings.sh
```

A.6 V7 poison message. Use the drill script delivered by #459 if it exists. Otherwise publish one malformed envelope to the topic with the routing attribute the notification subscription filters on (read the filter policy first; a message without it is filtered out and never reaches the queue):

```
TOPIC=$(aws sns list-topics --query "Topics[?ends_with(TopicArn, ':blocbuy-swe5001-domain-
events')].TopicArn" --output text)
aws sns list-subscriptions-by-topic --topic-arn "$TOPIC"     # find the notification subscription
aws sns get-subscription-attributes --subscription-arn <arn> --query 'Attributes.FilterPolicy'
aws sns publish --topic-arn "$TOPIC" --message '{"malformed":true}' \
  --message-attributes '{"eventType":{"DataType":"String","StringValue":"PaymentSucceeded"}}'
```

Add any further attribute the filter policy requires. With a 60 s visibility timeout and `maxReceiveCount` 5, expect the DLQ after about 5–6 minutes and the alarm a few minutes later.

A.7 V3 load (load machine, POSIX shell, from the repository root). Ramp: 20% → 100% of `KNEE_RPS` in 5 min, to 300% in 10 min, hold 10 min, down 5 min (30 min total). The k6 preflight accepts only a stable one-task commerce deployment with scaling bounds 1..6 (`perf/k6/lib.mjs`), so the standing floor of 2 is lowered first, as on 19 October.

(a) About 16:05, Liu Jin, signed in through Identity Center with administrator rights: lower the commerce floor to 1 and wait for a stable 1/1 deployment.

```
aws application-autoscaling register-scalable-target --service-namespace ecs \
  --resource-id service/blocbuy-swe5001/commerce-svc --scalable-dimension ecs:service:DesiredCount
\
```

```
  --min-capacity 1 --max-capacity 6
aws ecs update-service --cluster blocbuy-swe5001 --service commerce-svc --desired-count 1
aws ecs wait services-stable --cluster blocbuy-swe5001 --services commerce-svc
```

(b) 16:23–16:27, load machine: capture the V3 preflight in the clean reviewed checkout whose `HEAD` becomes `TEST_SHA`, with the operator's AWS environment (`perf/k6/README.md`). It is read-only, and k6 rejects it if it is more than five minutes old when the load starts.

```
results=$(mktemp -d); chmod 700 "$results"
umask 077
bash infra/swe5001/scripts/http-preflight.sh v3 > "$results/preflight.json"
```

(c) 16:28, on `K6 GO`: start the load.

```
export BASE_URL=https://d3lvq02o2e7tcn.cloudfront.net CONFIRM_TARGET=https://d3lvq02o2e7tcn.
cloudfront.net
export TEST_SHA=$(git rev-parse HEAD) IMAGE_SHA='<deployed commerce source sha>'
export EXPECTED_IMAGE_DIGEST='sha256:<deployed commerce digest>' RUN_LABEL=20261021-v3-presentation
docker run --rm --user "$(id -u):$(id -g)" \
  --mount "type=bind,source=$PWD/perf/k6,target=/scripts,readonly" \
  --mount "type=bind,source=$results,target=/results" \
  --env BASE_URL --env CONFIRM_TARGET --env TEST_SHA --env IMAGE_SHA --env EXPECTED_IMAGE_DIGEST --
env RUN_LABEL \
  grafana/k6:1.6.1@sha256:a5ad6bc089a08d77c3ec49f3db8c6fa7a148e4073efcac44c675dbaf3568d8e1 run \
  --env MODE=v3 --env TOPOLOGY=distributed --env KNEE_RPS=200 \
  --env PREFLIGHT_PATH=/results/preflight.json --env SUMMARY_PATH=/results/summary.json \
  --out json=/results/samples.jsonl /scripts/browse.js
```

After the talk, A.9 `down` removes the whole runtime, floor included. If the environment is not torn down that day, set the commerce floor back to 2:

```
aws application-autoscaling register-scalable-target --service-namespace ecs \
  --resource-id service/blocbuy-swe5001/commerce-svc --scalable-dimension ecs:service:DesiredCount
\
  --min-capacity 2 --max-capacity 6
```

A.8 V4 burst (not used in the demo; requires the reviewed `v4_demo_enabled=true` deployment).

```
export V4_RUN_LABEL=v4-20261021-presentation V4_RECIPIENT_ID=<already-active synthetic identity
uuid>
export EXPECTED_IMAGE_DIGEST=sha256:<commerce digest>
EXPECTED_NOTIFICATION_DIGEST=sha256:<notification digest>
export CONFIRM_V4_SEED=2000 V4_RESULTS_DIR=<new empty private directory outside the checkout>
bash infra/swe5001/scripts/v4-burst.sh
```

Restore `v4_demo_enabled=false` with a reviewed apply after the talk (or let `down` remove the runtime).

A.9 Tear down and verify

```
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=down
gh workflow run swe5001-env.yml --repo PPsyrius/iss-groupbuy-team8 --ref prod -f action=status
```

B Appendix — backup slides (Q&A only)

The backups follow slide 24 in the HTML deck and the PDF. In the deck, press **B** for the index (BX) and click a row; slide 24 stays up otherwise. Each backup gives the AD and AISS IDs in the Q&A answers a slide to point to.

ID	Slide	Answers	Owner	What to say
B1a	Logical decisions (AD-02 to AD-06)	Why did you choose X over Y? (logical)	Eric	Read the decision, then the reason. AD-02 is slide 7; AD-04 to AD-06 are the cards on slide 8.
B1b	Physical decisions (AD-07 to AD-14)	Why Fargate, a NAT instance, one database server?	Liu Jin	Costs are AWS list prices for ap-southeast-1; the US$125/month always-on figure is the proposal estimate.
B2	Known limitations and the way out	Single points of failure, a large commerce service	Liu Jin	Name the limitation plainly, then the resolution path. Single-zone database and cache is AISS-01.
B3	Context map: where service boundaries may go	What does DDD actually change?	Eric	Four contexts share one service because they share transactions. Anti-corruption layers sit in front of Google and Stripe; notification only hears published events.
B4	Aggregates and their life cycles	How do a team and an order change state?	Eric	A team can move only once, from OPEN. FAILED and CANCELED are allowed by the status table but nothing in the code triggers them: say so if asked.
B5	Layers, and tests that keep boundaries in place	How do you stop boundaries eroding?	Eric	18 architecture rules run with the unit tests, so a call across a boundary fails the build.
B6	Joining a team by paying, step by step	Show me the payment path, not expiry	Yorik	Signature is checked outside the transaction. One transaction then deduplicates, locks in a fixed order, admits the member, saves the order, takes stock and writes the outbox; the active user is re-checked inside it.
B7	Technology and services	Exact services, sizes and versions	Liu Jin	Read only the row that was asked about.
B8	Everything the real cloud taught us	What else went wrong?	Pan	Ten findings in order. The last open one is the paused-scaling repair, which has no fix yet.
B9	Infrastructure as code, and the actual bill	Stacks, lifetimes and real cost	Pan	The whole-account bill is mostly the always-on development server; this project's runtime cost US$0.49.
B10	API access for producers and consumers	Platform engineering: robust API access	Eric	Be upfront about what is not built: no API gateway, per-client keys or rate limiting; internal endpoints rely on network rules.
B11	Analytics: what the platform measures	Analytics to improve the platform	Lim	Operational analytics are in use. Business analytics are possible from data we already store (share invites and referral credits), but no reports are built yet.
B12	Scale-out and resilience results	Evidence behind slides 5, 14 and 17	Lim	Never upgrade a local result to an AWS one. Anything not run on AWS stays "not run on AWS" with its local result.
B13	Logical view, simplified	Plainer companion to slide 9	Eric	Same content as slide 9 without UML notation.
B14	Physical view, simplified	Plainer companion to slide 13	Liu Jin	Same content as slide 13 without UML notation.

B15	Team expiry, numbered steps	Plainer companion to slide 16	Yorik	Use the slide 16 wording; the step numbers match.